

FinPilot — Comprehensive Interview Q&A

Audience: Intuit SDE Interview (2 YOE)

Format: Interviewer question → Your answer

Depth: Junior to mid-level, calibrated for real interview pushback

Table of Contents

1. [Project Walkthrough](#)
 2. [Architecture & Design Patterns](#)
 3. [Technology Choices \(Why X Over Y\)](#)
 4. [Database Design](#)
 5. [API Design & REST Practices](#)
 6. [Authentication & Authorization](#)
 7. [Fintech Security & Data Protection](#)
 8. [Financial Data Guardrails](#)
 9. [AI/ML Integration](#)
 10. [CSV Import & Data Ingestion](#)
 11. [Multi-Currency Handling](#)
 12. [Error Handling & Resilience](#)
 13. [Scalability & Performance](#)
 14. [Frontend Architecture](#)
 15. [Testing Strategy](#)
 16. [DevOps, CI/CD & Infrastructure](#)
 17. [Monitoring & Observability](#)
 18. [Trade-offs & Honest Limitations](#)
 19. [Behavioral & Scenario Questions](#)
 20. [Intuit-Specific Questions](#)
-

1. Project Walkthrough

Q: Walk me through your project in 2 minutes.

A: FinPilot is a personal finance intelligence platform I built end-to-end. Users can track income and expenses in multiple currencies, set budgets against categories, and get AI-powered savings recommendations. The core flows are:

1. User signs up → system seeds 12 default spending categories
2. Transactions are added manually or via CSV import (bank statement upload)
3. Every transaction amount is converted to the user's base currency at write time, locking in the exchange rate
4. Dashboard aggregates spending by category, shows trends, and compares budget vs. actual
5. AI engine analyzes 90 days of spending history, detects patterns like recurring subscriptions or dining spikes, and returns personalized savings recommendations
6. Scheduled PDF/CSV reports are emailed monthly via cron jobs

The stack is **Node.js + Express + TypeScript** on the backend, **React + Vite + TailwindCSS** on the frontend, **PostgreSQL via Prisma ORM**, all deployed on **AWS ECS Fargate** with **Aurora Serverless**, fronted by **CloudFront CDN**. Infrastructure is managed with **Terraform**.

Q: Why did you build this? What problem does it solve?

A: Most budgeting apps either lack multi-currency support or charge for AI insights. I wanted to build something that handles real-world complexity — people travel, earn in different currencies, have subscriptions across countries. FinPilot converts everything to a base currency at the time of transaction, so your monthly reports are always consistent. The AI component doesn't just show you charts — it gives actionable advice like "Your dining spend increased 40% this month, you could save ₹3,200 by cooking 2 more meals a week."

Q: What was the hardest technical challenge you faced?

A: Multi-currency handling. The naive approach is to convert all amounts on read using today's exchange rate — but that means your January report changes every day as rates fluctuate. I solved this with **write-time conversion**: when a transaction is created, I fetch the current exchange rate, multiply to get the base currency amount, and store both the original amount AND the converted amount with the locked exchange rate. This makes all aggregation queries use only `baseCurrencyAmount`, which is immutable and consistent.

The second challenge was making the AI pipeline resilient. LLM calls can fail, return malformed JSON, or hallucinate numbers. I built a **deterministic rule-engine fallback** — if the LLM fails or returns invalid data, the system generates recommendations using hard-coded rules (e.g., "if dining > 30% of income, recommend reduction"). Users always get useful output regardless of API availability.

Q: What's the scale of this application?

A: Currently it's a personal/small-team project. Stress tests showed it handles ~39 requests/second on minimal infrastructure (0.25 vCPU, 512 MB). The architecture supports horizontal scaling — ECS auto-scaling between 2-10 tasks, Aurora auto-scales from 0.5 to 4 ACUs. With a single config change (bumping CPU to 1 vCPU), it can comfortably handle 50+ concurrent users. The important thing is the **architecture is scale-ready** — stateless services, no in-memory sessions, database connection pooling via Prisma — even though I haven't needed production-scale yet.

Q: What would you do differently if you started over?

A: Three things:

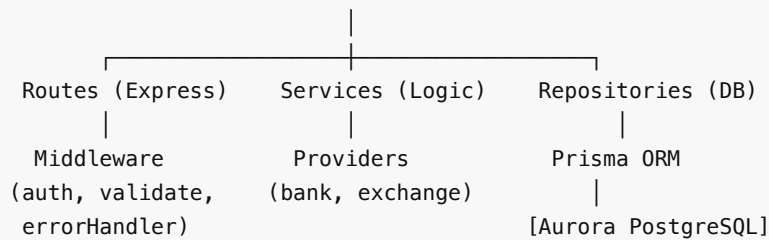
1. **Event-driven CSV import** — right now it processes row-by-row synchronously. For large files, I'd use a queue (SQS) so the API returns immediately and processing happens async.
 2. **Separate read/write models for dashboard** — the dashboard queries aggregate on the fly. With materialized views or a pre-computed summary table, dashboard loads would be constant-time regardless of transaction count.
 3. **Token rotation** — I use a single JWT secret for both access and refresh tokens. In production, I'd use separate keys, key rotation, and consider asymmetric signing (RS256) so services could verify tokens without sharing the secret.
-

2. Architecture & Design Patterns

Q: Describe your system architecture.

A: It's a **modular monolith** with a clean 3-layer architecture:

[React SPA] → [CloudFront CDN] → [ALB] → [ECS Fargate]



- **Routes** handle HTTP concerns: parsing, validation, response formatting
- **Services** contain all business logic: currency conversion, AI analysis, report generation
- **Repositories** abstract database access: every Prisma query is wrapped in a repository method

This separation means I can swap PostgreSQL for another database by only changing the repository layer, or replace Express with Fastify by only changing the route layer.

Q: Why a modular monolith instead of microservices?

A: For a team of one or two engineers, microservices add operational overhead that outweighs the benefits:

Concern	Microservices	Modular Monolith
Deployment	Multiple pipelines	One pipeline
Communication	Network calls (latency + failure modes)	Function calls (nanoseconds)
Data consistency	Distributed transactions / Saga	Single DB transactions
Debugging	Distributed tracing required	Stack trace in one process
Cost	Multiple containers, service mesh	One container

My services are already cleanly separated — `TransactionService`, `BudgetService`, `SavingsAIService` don't share internal state. If I needed to scale one independently (say AI recommendations are expensive), I could extract just that service behind a queue without redesigning the entire system.

The framing I use: **designed for current scale, architected for future scale.**

Q: Explain the Repository pattern in your project.

A: Every database operation goes through a repository class rather than calling Prisma directly from services. For example:

- `TransactionRepository` has methods like `create()`, `findById()`, `sumByCategory()`
- `UserRepository` has `findByEmail()`, `create()`, `softDelete()`

Why:

1. **Testability** — services can be tested with mock repositories without needing a database
2. **Single responsibility** — query optimization is contained in one place
3. **Swappability** — I could replace Prisma with raw SQL or another ORM by only changing repository implementations

There's also a `BaseRepository` with shared CRUD patterns that other repositories extend.

Q: Explain the Provider pattern you use.

A: The Provider pattern abstracts external integrations behind a common interface. I use it for:

1. **Exchange rates** — `ExchangeRateProvider` interface with `getRate(base, target)` .

Implementations:

- `ApiExchangeRateProvider` — calls external API
- `DatabaseExchangeRateProvider` — uses cached rates from DB
- `FixedExchangeRateProvider` — hardcoded fallback rates
- `FallbackRateProvider` — chains providers: API → DB → Fixed

2. **Bank sync** — `BankProvider` interface with standardized methods

This means if my exchange rate API goes down, the system automatically falls through to the DB cache and then to hardcoded rates. Users never see an error — they might get slightly stale rates, which is a reasonable degradation for a personal finance app.

Q: What is the middleware chain in your Express app?

A: Requests flow through middleware in this order:

1. **CORS** — configured for frontend origin, credentials allowed
2. **Helmet** — sets security headers (X-Content-Type-Options, X-Frame-Options, HSTS, etc.)
3. **Cookie parser** — parses JWT from HttpOnly cookies
4. **JSON body parser** — with size limit
5. **Request logger** — logs method, URL, status code, response time
6. **Route-level authenticate** — verifies JWT, attaches `user` to request
7. **Route-level validate** — runs Zod schema against body/query/params
8. **Route handler** — wrapped in `asyncHandler` to catch Promise rejections
9. **Error handler** — global, catches all errors, formats response based on error type

The `authenticate` middleware reads the token from `req.cookies.accessToken` first, then falls back to the `Authorization: Bearer` header. This supports both browser (cookie) and API (header) authentication.

Q: How does your error handling work?

A: I have a custom error hierarchy:

```
AppError (base, statusCode + isOperational flag)
├─ ValidationError (400, carries field-level errors object)
├─ NotFoundError (404, includes resource name + ID)
├─ AuthError (401)
├─ ForbiddenError (403)
└─ ConflictError (409)
```

Flow:

1. Service throws a typed error (e.g., `throw new NotFoundError("Transaction", id)`)
2. `asyncHandler` catches it and forwards to Express error middleware via `next(err)`
3. Global `errorHandler` checks `instanceof` :

- `ValidationError` → 400 with `{ success: false, message, errors: { field: ["messages"] } }`
- `AppError` → uses `err.statusCode` with clean message
- Unknown errors → 500 with generic message in production, real message in development

4. Unknown errors are also logged with full stack trace via Winston

The key insight: `isOperational` flag distinguishes expected business errors (user not found) from unexpected bugs (null reference). In production, only operational errors show their message to the client.

Q: What design patterns do you use?

A:

Pattern	Where	Why
Repository	All DB access	Decouple business logic from data layer
Provider / Strategy	Exchange rates, bank sync	Swap implementations without changing consumers
Chain of Responsibility	FallbackRateProvider	Try multiple sources in order
Middleware	Express pipeline	Cross-cutting concerns (auth, logging, validation)
Factory	Default category seeding	Create 12 categories with consistent structure on registration
Facade	DashboardService	Single method aggregates data from multiple repositories
Template Method	BaseRepository	Common CRUD, specialized queries in subclasses

3. Technology Choices (Why X Over Y)

Q: Why Node.js + Express over Python/Django or Java/Spring?

A:

- **JavaScript everywhere** — same language for frontend and backend reduces context switching, shared types via TypeScript
- **Non-blocking I/O** — a finance app is I/O heavy (database queries, API calls to exchange rate providers, AI APIs). Node's event loop handles concurrent I/O efficiently without threads
- **Express** specifically because it's minimal — I add exactly what I need. Django is batteries-included which is great for content sites but overkill for a REST API. Spring has heavy ceremony and slow startup — ECS Fargate charges for running time, so cold start matters
- **Ecosystem** — Prisma, Zod, Helmet, cookie-parser, multer — everything I needed had mature Node libraries

Follow-up — wouldn't Go or Rust be faster?

Yes, for CPU-bound work. But my bottleneck is bcrypt hashing (CPU) and database queries (I/O). Going to

Go would save maybe 10ms on JSON serialization but add development time. Node is fast enough — stress tests showed 25ms p95 latency for API calls.

Q: Why TypeScript over plain JavaScript?

A:

- **Type safety** — in a finance app, a type mismatch (string vs number for amount) could cause real monetary errors. TypeScript catches these at compile time
 - **IDE support** — autocomplete, refactoring, inline docs. I can refactor a repository method signature and TypeScript tells me every service that needs updating
 - **Self-documenting** — types like `SupportedCurrency = "USD" | "EUR" | "GBP" | "INR"` act as documentation
 - **Zod + TypeScript** — my validation schemas infer TypeScript types, so the runtime validation and compile-time types are always in sync
-

Q: Why PostgreSQL over MongoDB or MySQL?

A:

- **Financial data is relational** — transactions belong to categories, categories belong to users, budgets link users to categories for a month. These are natural foreign-key relationships
- **ACID transactions** — when I create a transaction, I need to update the exchange rate log in the same atomic operation. MongoDB's document transactions are less mature
- **Decimal precision** — PostgreSQL's `DECIMAL(15,2)` is exact. MongoDB stores numbers as IEEE 754 doubles, which can have floating-point errors on financial amounts
- **Advanced querying** — I use `GROUP BY category, SUM(baseCurrencyAmount)` for dashboard. PostgreSQL's query planner handles these aggregations efficiently with proper indexes
- **JSON support** — for the AI recommendations, I store structured JSON in a `Json` column. PostgreSQL gives me relational WHERE it matters and document-style WHERE flexibility helps

Follow-up — what about MySQL?

MySQL would work. PostgreSQL has better JSON support, better default transaction isolation (Read Committed vs MySQL's Repeatable Read which can cause phantom reads), and Aurora PostgreSQL Serverless integrates better with my infrastructure.

Q: Why Prisma over Sequelize, TypeORM, or raw SQL?

A:

- **Type-safe queries** — Prisma generates TypeScript types from the schema. When I write `prisma.transaction.findMany({ where: { userId } })`, the return type is fully typed including relations
- **Schema-as-code** — `schema.prisma` is the single source of truth for database structure. Migrations are auto-generated from schema changes
- **Prevention of SQL injection** — Prisma parameterizes all queries. I never write raw SQL strings
- **Developer experience** — Prisma Studio for debugging, auto-completion for every query

Follow-up — doesn't Prisma add overhead?

Yes, there's a query engine process. For my scale, the overhead is negligible (~2-5ms). If I were doing thousands of queries per second, I'd consider raw SQL for hot paths while keeping Prisma for less critical code.

Why not TypeORM?

TypeORM decorators are runtime-heavy and not as type-safe — you can write queries that TypeScript thinks are correct but fail at runtime. Prisma's generated client prevents this.

Q: Why React over Angular or Vue?

A:

- **Component model** — React's "everything is a component" maps cleanly to financial UI: TransactionList, BudgetCard, DashboardChart are all self-contained
 - **Ecosystem size** — TanStack Query for data fetching, Recharts for charts, Zustand for state — all first-class React libraries
 - **Intuit uses React** — TurboTax and Mint are built with React, so it aligns with your stack
 - **Hooks** — custom hooks like `useTransactions()`, `useDashboard()` encapsulate API + caching logic. Components stay pure rendering
-

Q: Why Vite over Create React App or Webpack?

A:

- **Speed** — Vite uses esbuild for dev and Rollup for production. Dev server starts in <300ms vs 10+ seconds with CRA
 - **HMR** — Hot Module Replacement is instant. When I edit a component, the browser updates without full page reload
 - **CRA is deprecated** — React team recommends Vite or Next.js now
 - **Simple config** — my `vite.config.ts` is ~15 lines. Equivalent Webpack config would be 100+
-

Q: Why TanStack Query over Redux or SWR?

A:

- **Server state ≠ client state** — finances are server-authoritative. TanStack Query treats API data as a cache: fetch, cache, invalidate, refetch. This is exactly what I need
- **Automatic cache invalidation** — when I create a transaction, `queryClient.invalidateQueries(["transactions"])` automatically refetches the list
- **Loading/error states** — built-in `isLoading`, `isError`, `data` triple. No manual state management
- **Background refetch** — when the user returns to the dashboard tab, stale data is silently refreshed

Redux would require writing action creators, reducers, selectors, middleware (thunk/saga) just to fetch and cache API data. TanStack Query does all of that in 3 lines.

SWR is similar but TanStack Query has better mutation support, pagination, and devtools.

Q: Why Zustand over Redux for client state?

A: I use TanStack Query for server state (API data). Zustand handles the small amount of true client state — user preferences, UI state like sidebar open/closed, selected currency. Zustand is:

- **Tiny** (4KB) vs Redux (7KB + middleware)
 - **No boilerplate** — a store is 5 lines, no actions/reducers/dispatch
 - **Works outside React** — I can read store state in utility functions
-

Q: Why TailwindCSS over styled-components or CSS Modules?

A:

- **Utility-first** — `className="bg-white rounded-lg shadow-sm p-6"` is faster than creating a styled component or writing CSS classes
 - **Consistent design system** — spacing (`p-4` , `m-2`), colors (`text-gray-600`), typography are all standardized
 - **Purging** — unused classes are removed in production build. Final CSS is ~10KB instead of 200KB+
 - **No runtime cost** — styled-components processes CSS at runtime. Tailwind is pure CSS, zero JS overhead
-

Q: Why ECS Fargate over EC2, Lambda, or Kubernetes?

A:

- **Fargate over EC2** — no instance management, patching, or capacity planning. I define CPU/memory and AWS handles the rest
- **Fargate over Lambda** — my app has persistent connections (database connection pool), cron jobs, and WebSocket potential. Lambda's stateless, short-lived model doesn't fit
- **Fargate over Kubernetes** — EKS is overkill for a single service. I'd need a control plane, node groups, YAML manifests, Helm charts. Fargate gives me container orchestration without the operational burden

Cost: 0.25 vCPU + 512MB costs ~\$10/month. Same on EC2 (t3.micro) would be \$8 but I'd manage OS updates, security patches, and auto-scaling groups myself.

Q: Why Aurora Serverless over RDS or self-managed PostgreSQL?

A:

- **Auto-scaling** — Aurora Serverless v2 scales from 0.5 to 4 ACUs based on load. I don't pay for capacity I'm not using
 - **Managed** — automated backups (7-day retention), point-in-time recovery, encryption at rest with KMS
 - **High availability** — Aurora replicates 6 copies of data across 3 AZs automatically
 - **Cost** — for variable workloads (personal finance app with evening/weekend spikes), serverless is cheaper than provisioned
-

Q: Why CloudFront for the frontend?

A:

- **Edge caching** — static assets (JS, CSS, images) served from the nearest edge location. Users in Mumbai get assets from the Mumbai edge, not from us-east-1
 - **SPA routing** — custom error response: 403/404 → `/index.html` with 200 status. This makes React Router's client-side routing work
 - **HTTPS** — CloudFront provides free SSL via ACM certificate
 - **Origin Access Control** — S3 bucket is private. Only CloudFront can read from it. No direct S3 URL access
 - **API proxying** — `/api/*` routes to ALB origin, everything else to S3. Single domain, no CORS issues in production
-

Q: Why JWT over sessions?

A:

- **Stateless** — the server doesn't need to store session data. Each request carries its own authentication proof
- **Horizontal scaling** — any ECS task can verify the token. With sessions, I'd need sticky sessions or a shared session store (Redis)
- **Mobile-ready** — if I add a mobile app, JWT works as-is. Sessions are browser-specific

But I store JWTs in HttpOnly cookies, not localStorage. This gives me the stateless benefit of JWT with the XSS protection of cookies. The token is never accessible to JavaScript.

Q: Why Zod over Joi or class-validator?

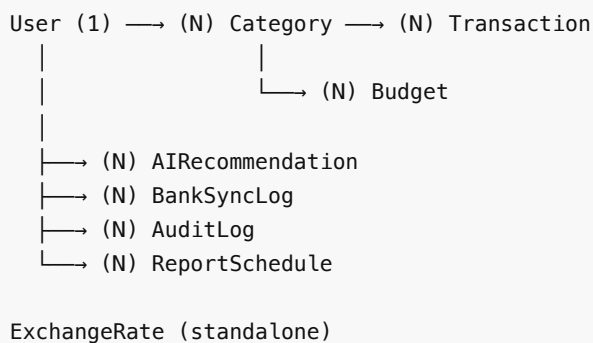
A:

- **TypeScript-first** — Zod schemas infer TypeScript types. `z.infer<typeof createTransactionSchema>` gives me a type that's always in sync with validation
 - **Composable** — I can merge, extend, and pick from schemas. `updateTransactionSchema` reuses `createTransactionSchema.partial()`
 - **Small** — ~13KB vs Joi's ~150KB
 - **No decorators** — class-validator requires classes with decorators. Zod works with plain objects, which matches Express's `req.body`
-

4. Database Design

Q: Walk me through your database schema.

A: Nine models, all connected:



Key models:

- **User** — email, passwordHash, baseCurrency, soft-delete via `deletedAt`
 - **Transaction** — amount, currency, `baseCurrencyAmount`, `exchangeRate` (locked at creation), type (INCOME/EXPENSE), tags (String array)
 - **Category** — hierarchical (`parentId` self-reference), type, color, icon, `isDefault` flag
 - **Budget** — links user + category + month, amount in base currency
 - **AIRecommendation** — stores LLM output as JSON, category, priority, estimated savings, status (PENDING/ACCEPTED/DISMISSED)
-

Q: Why CUID over UUID or auto-increment for primary keys?

A:

- **CUID over auto-increment** — auto-increment IDs are sequential, which leaks information (user can guess `GET /transactions/1042` exists). CUIDs are random-looking
 - **CUID over UUID** — CUIDs are shorter (25 chars vs 36), k-sortable (roughly time-ordered), and have better index locality than UUIDv4
 - **No enumeration attacks** — an attacker can't iterate through IDs to scrape data
-

Q: Why `Decimal(15,2)` for monetary amounts?

A: IEEE 754 floating-point cannot precisely represent most decimal fractions:

```
0.1 + 0.2 = 0.30000000000000004 // in JavaScript
```

In finance, this is unacceptable. A billing error of \$0.01 on 100,000 transactions is \$1,000. `Decimal(15,2)` stores exact decimal values — `0.1 + 0.2 = 0.30` always.

- **15 digits total** — supports amounts up to `9,999,999,999,999.99` (99 trillion, handles any personal finance amount)
 - **2 decimal places** — matches standard currency precision
 - **Prisma returns Decimal as string** — I parse to number only at the API response boundary, keeping DB operations exact
-

Q: Explain your indexing strategy.

A:

```
@@index([userId, date])           // Transaction: filter by user, sort by date
@@index([userId, categoryId])     // Transaction: spending by category
@@index([userId])                 // Budget, Category, etc.
@@unique([userId, categoryId, month]) // Budget: one budget per category per month
@@unique([baseCurrency, targetCurrency]) // ExchangeRate: quick lookup
@@index([userId, deletedAt])      // User: soft-delete queries
```

Composite index order matters: `[userId, date]` serves queries that filter by `userId` AND sort by date. The database can scan the index rather than doing a full table scan. I put `userId` first because every query is user-scoped (multi-tenant).

Why not index everything?

Indexes speed up reads but slow down writes (every INSERT/UPDATE must update the index). I only index columns that appear in WHERE, ORDER BY, or JOIN conditions.

Q: How does soft-delete work?

A: Users have a `deletedAt DateTime?` field. When a user "deletes" their account:

- `deletedAt` is set to `new Date()` instead of actually deleting the row
- All queries filter `WHERE deletedAt IS NULL`

Why soft delete for Users?

- **Data retention** — financial transaction history may need to be preserved for compliance
- **Account recovery** — user can contact support to reactivate
- **Referential integrity** — hard-deleting a user while transactions reference them would violate foreign keys

For transactions and categories, I use **hard delete with cascade** — if a category is deleted, its transactions are reassigned or deleted explicitly through the service layer to maintain data integrity.

Q: Why store `baseCurrencyAmount` and `exchangeRate` on every transaction?

A: This is the **write-time conversion** pattern:

```
Transaction {
  amount: 100.00      // original amount
  currency: "EUR"     // original currency
  baseCurrencyAmount: 110.50 // converted at creation time
  exchangeRate: 1.105   // rate locked at creation
}
```

Without this: Every dashboard query would need to join with exchange rates and multiply. Rates change daily, so your January spending total would be different tomorrow.

With this: `SUM(baseCurrencyAmount) WHERE date BETWEEN '2024-01-01' AND '2024-01-31'` is a simple, fast query that always returns the same result. The aggregation column is pre-computed and immutable.

Trade-off: Slightly more storage per row (two extra Decimal columns). Worth it for query simplicity and financial accuracy.

Q: How do you prevent duplicate transactions in bank sync?

A: The `BankSyncService` uses a **composite deduplication key**: `description + date + amount`. Before inserting, it queries:

```
WHERE userId = ? AND description = ? AND date = ? AND amount = ?
```

If a match exists, the row is skipped. This prevents double-importing when a user syncs the same bank statement twice.

Edge case: What if two genuinely different transactions have the same description, date, and amount? (e.g., two \$5.00 Starbucks charges on the same day)

Currently, the second would be skipped. A better solution would include a bank-provided transaction ID (if available) or a hash of additional fields.

5. API Design & REST Practices

Q: How are your APIs structured?

A: RESTful, resource-based, with consistent patterns:

```
POST    /api/auth/register    # Create user
POST    /api/auth/login       # Authenticate
```

GET	/api/transactions	# List (with pagination, filters)
POST	/api/transactions	# Create
GET	/api/transactions/:id	# Get one
PUT	/api/transactions/:id	# Update
DELETE	/api/transactions/:id	# Delete
POST	/api/transactions/import	# CSV import (custom action)
GET	/api/dashboard/summary	# Aggregated data
POST	/api/ai/recommendations	# Generate AI analysis

Patterns:

- All responses follow `{ success: boolean, data?: T, message?: string }`
- List endpoints support `?page=1&limit=20&sort=date&order=desc`
- Filters via query params: `?type=EXPENSE&categoryId=xxx&startDate=2024-01-01`
- 39 total endpoints across 8 route groups

Q: How do you handle pagination?

A: Offset-based pagination:

```
GET /api/transactions?page=1&limit=20
Response: { data: [...], total: 150, page: 1, limit: 20 }
```

Why offset over cursor?

For a personal finance app with <100K transactions per user, offset pagination works fine. Cursor-based is better at extreme scale (millions of rows) because OFFSET N requires scanning N rows. But for my use case, the index on `[userId, date]` makes OFFSET efficient.

Q: How do you validate API inputs?

A: Three-layer validation:

1. **Zod schemas** — defined in `validation.ts` for every endpoint. Example:

```
const createTransactionSchema = z.object({
  amount: z.number().positive(),
  currency: z.enum(["USD", "EUR", "GBP", "INR", ...]),
  categoryId: z.string().min(1),
  type: z.enum(["INCOME", "EXPENSE"]),
  description: z.string().min(1).max(500),
  date: z.string().datetime(),
  tags: z.array(z.string()).optional()
});
```

2. **validate middleware** — passes `req.body` (or `req.query` or `req.params`) through the Zod schema. On failure, creates a `ValidationError` with field-level error messages and calls `next(error)`
3. **Database constraints** — Prisma schema enforces NOT NULL, unique constraints, foreign keys as a final safety net

Why validate at the middleware level?

Services receive **pre-validated, correctly-typed data**. Business logic doesn't need defensive null checks. This is the "parse, don't validate" philosophy — once data passes Zod, it's guaranteed to match the TypeScript type.

Q: How do you handle file uploads (CSV import)?

A: Using `multer` middleware:

1. **multer** configured with memory storage (no temp files on disk) and file size limit
2. File arrives as `req.file.buffer`
3. Buffer piped through `csv-parser` (streaming — doesn't load entire file into memory)
4. Each row validated with Zod schema
5. Valid rows processed with category resolution (match by name or create new)
6. Results returned: `{ imported: 45, skipped: 3, errors: [{row: 12, reason: "Invalid amount"}] }`

Why memory storage over disk?

On ECS Fargate, the filesystem is ephemeral and limited. Memory storage avoids failed cleanup of temp files and is faster for small-to-medium CSVs (under 10MB).

6. Authentication & Authorization

Q: Walk me through your auth flow.

A:

Registration:

1. User submits email + password + name + baseCurrency
2. Zod validates input (email format, password min length)
3. Check if email already exists → `ConflictError` if yes
4. Hash password with **bcrypt (12 salt rounds)**
5. Create user in DB
6. Seed 12 default categories for the user
7. Generate JWT access token (15 min) and refresh token (7 days)
8. Set both as **HttpOnly, Secure, SameSite=Strict** cookies
9. Return user profile (never the password hash)

Login:

1. Find user by email → `AuthError` if not found
2. Compare password with `bcrypt` → `AuthError` if mismatch
3. Generate and set JWT cookies (same as registration)

Protected request:

1. `authenticate` middleware reads `accessToken` from cookies
 2. `verifyToken()` validates JWT signature and expiry
 3. Decoded payload (`{ userId, email }`) attached to `req.user`
 4. Route handler accesses `req.user.userId` to scope all DB queries
-

Q: Why HttpOnly cookies instead of localStorage for JWT?

A:

Threat	localStorage	HttpOnly Cookie
XSS	❌ Vulnerable — JS can read <code>localStorage.getItem("token")</code>	✅ Protected — cookie is invisible to JavaScript
CSRF	✅ Not vulnerable (token must be manually set in header)	⚠️ Partially vulnerable — mitigated with <code>SameSite=Strict</code>
Man-in-Middle	Depends on HTTPS	✅ Secure flag ensures cookie only sent over HTTPS

My setup:

- `HttpOnly: true` → JavaScript cannot access the token
- `Secure: true` → only sent over HTTPS
- `SameSite: Strict` → cookie not sent on cross-origin requests, preventing CSRF

This combo protects against both XSS and CSRF, which are the two main web attack vectors.

Q: Why bcrypt with 12 rounds? Why not argon2?

A:

- **bcrypt (12 rounds)** produces ~250ms hash time per password. This makes brute-force attacks impractical — an attacker trying 1 million passwords would need ~69 hours
 - **12 rounds is the current industry recommendation.** 10 is the minimum, 12-14 is standard
 - **Why not argon2?** Argon2 is newer and considered superior (it's memory-hard, not just CPU-hard). I chose bcrypt because:
 - It's battle-tested (20+ years in production)
 - Better library support in Node.js
 - For a personal finance app (not handling millions of logins), bcrypt at 12 rounds is sufficient
 - If I were building for Intuit-scale, I'd use argon2id with tuned memory/parallelism parameters
-

Q: How do you handle authorization (not authentication)?

A: Every database query is **scoped by** `userId` :

```
// TransactionRepository
findByUserId(userId: string, filters) {
  return prisma.transaction.findMany({
    where: { userId, ...filters }
  });
}
```

There's no way to access another user's data because:

1. `userId` comes from the verified JWT payload, not from request parameters
2. Every repository method takes `userId` as the first argument

3. Even if an attacker guesses a transaction ID, the query `WHERE id = ? AND userId = ?` prevents access

This is **row-level security at the application level**. It's not as strong as database-level RLS (PostgreSQL policies), but it's consistent and hard to accidentally bypass due to the repository pattern.

Q: What are the weaknesses in your current auth?

A: (Being honest in an interview shows maturity)

1. **Single JWT secret** — both access and refresh tokens use the same `JWT_SECRET`. If it leaks, both are compromised. Fix: separate keys, or asymmetric signing (RS256)
 2. **No token revocation** — if a user logs out, the token is still valid until expiry. Fix: maintain a token blacklist in Redis, or implement short-lived tokens with refresh rotation
 3. **No rate-limiting on login** — an attacker could brute-force passwords. Fix: rate limiter middleware (e.g., `express-rate-limit`) with exponential backoff
 4. **No MFA** — single-factor auth. For production fintech, TOTP (Google Authenticator) or SMS verification would be important
 5. **Password rules are basic** — Zod validates minimum length but not complexity. This is a UX/security trade-off
-

7. Fintech Security & Data Protection

Q: What security measures do you have for handling financial data?

A: Security at each layer:

Network:

- HTTPS everywhere — CloudFront terminates TLS, ALB uses HTTPS listener
- VPC isolation — database is in private subnets, no public IP
- Security groups — only ALB can talk to ECS, only ECS can talk to Aurora
- NAT Gateway — outbound traffic from private subnets goes through NAT, no inbound

Application:

- Helmet.js — sets 11 security headers: CSP, X-Frame-Options, HSTS, X-Content-Type-Options, etc.
- CORS whitelist — only the frontend origin is allowed
- Input validation — Zod schemas on every endpoint prevent injection
- Parameterized queries — Prisma prevents SQL injection by design
- HttpOnly/Secure/SameSite cookies — prevent XSS, CSRF, MITM
- Error sanitization — production errors never leak stack traces or internal details

Data:

- Encryption at rest — Aurora uses AWS KMS encryption
- Encryption in transit — TLS 1.2+ for all connections (Aurora enforces `ssl=true`)
- Password hashing — bcrypt with 12 rounds (one-way, not reversible)
- No plaintext secrets — all credentials stored in AWS Secrets Manager
- Secrets injected at runtime — ECS task definition references Secrets Manager ARNs, not env vars

Infrastructure:

- Principle of least privilege — ECS task role only has permissions it needs (Secrets Manager read, CloudWatch write)

- Deletion protection — Aurora has `deletion_protection = true` in production
- Automated backups — 7-day retention with point-in-time recovery

Q: How would you ensure PCI DSS compliance?

A: PCI DSS applies when handling card numbers. FinPilot doesn't store credit card numbers directly — it processes transaction descriptions and amounts — so we're outside PCI scope for card storage. However, the principles still apply:

What I already do (PCI-aligned):

- Encrypt data at rest (KMS) and in transit (TLS)
- Unique user IDs with strong authentication
- No sensitive data in logs (Winston logger doesn't log request bodies)
- Restricted network access (private subnets, security groups)
- Audit logging via `AuditLog` model

What I'd add for full PCI compliance:

- Web Application Firewall (WAF) — CloudFront supports AWS WAF
- Vulnerability scanning — regular dependency audits (`npm audit`)
- Penetration testing schedule
- Formal access control policy
- Log retention and monitoring (currently CloudWatch, would add SIEM)

Key point for interview: "We don't store card numbers so we're mostly out of PCI scope, but we still follow PCI principles for defense-in-depth."

Q: How do you protect against the OWASP Top 10?

A:

Risk	How FinPilot Addresses It
A01: Broken Access Control	Every query scoped by <code>userId</code> from JWT. Repository pattern ensures consistent enforcement.
A02: Cryptographic Failures	<code>bcrypt</code> for passwords, KMS encryption at rest, TLS in transit. No plaintext secrets.
A03: Injection	Prisma ORM parameterizes all queries. <code>Zod</code> validates all inputs. No raw SQL.
A04: Insecure Design	Threat modeling: JWT in <code>HttpOnly</code> cookies, not <code>localStorage</code> . Rate fallbacks prevent service degradation.
A05: Security Misconfiguration	Helmet.js headers, CORS whitelist, production error messages sanitized, Terraform-managed infrastructure.
A06: Vulnerable Components	<code>npm audit</code> for known vulnerabilities. Dependabot for automated updates.
A07: Auth Failures	<code>bcrypt</code> (12 rounds), short-lived access tokens (15 min), <code>SameSite=Strict</code> cookies.

A08: Data Integrity	Zod schema validation on all inputs. Prisma type checking. Write-time currency conversion is immutable.
A09: Logging Failures	Winston structured logging, CloudWatch alerts, request logger middleware on every request.
A10: SSRF	No user-supplied URLs fetched server-side. Exchange rate API URLs are hardcoded constants.

Q: How is user data protected at rest and in transit?

A:

At rest:

- Aurora PostgreSQL uses **AWS KMS encryption** — every data page, log file, and snapshot is encrypted with AES-256
- S3 buckets use **SSE-S3** (server-side encryption) for static assets
- Secrets Manager encrypts all secrets with KMS
- Backups are encrypted (Aurora inherits encryption from the source cluster)

In transit:

- **TLS 1.2+** between:
 - Client ↔ CloudFront (ACM certificate)
 - CloudFront ↔ ALB (HTTPS origin policy)
 - ALB ↔ ECS containers (can be HTTP within VPC, or HTTPS with self-signed certs)
 - ECS ↔ Aurora (PostgreSQL `sslmode=require`)
 - ECS ↔ OpenAI API (HTTPS)
 - ECS ↔ Exchange Rate API (HTTPS)

Data minimization:

- I don't store raw bank credentials — bank sync uses a provider/token model
- Password hashes are one-way — even a database breach doesn't reveal passwords
- AI recommendations don't send PII to OpenAI — only aggregated spending data (categories + amounts, no names or account numbers)

Q: What data do you send to OpenAI? Any privacy concerns?

A: The AI pipeline sends only **aggregated financial data**:

```
{
  "monthlyIncome": 5000,
  "categoryBreakdown": [
    { "category": "Dining", "amount": 850, "percentOfIncome": 17 },
    { "category": "Subscriptions", "amount": 120, "percentOfIncome": 2.4 }
  ],
  "trends": [
    { "category": "Dining", "trend": "increasing", "changePercent": 40 }
  ]
}
```

What is NOT sent:

- User name, email, or any PII
- Individual transaction descriptions (which could contain merchant names)
- Bank account numbers or financial institution details

Privacy measures:

- Data is aggregated before sending — OpenAI sees spending categories and totals, not individual charges
 - OpenAI's API data policy: inputs are not used for model training (with API, as opposed to ChatGPT)
 - If I needed even more isolation, I could use Azure OpenAI with data residency guarantees
-

Q: How do you handle GDPR / data privacy?

A: While FinPilot isn't currently deployed in the EU, the architecture supports GDPR principles:

- **Right to access** — `GET /api/auth/profile` returns all stored user data
- **Right to deletion** — soft-delete allows account recovery, but I could add hard-delete for GDPR compliance
- **Data portability** — CSV export of all transactions (`GET /api/reports/export/csv`)
- **Consent** — no tracking pixels, no third-party analytics cookies
- **Data minimization** — I store only what's necessary for functionality
- **Encryption** — data at rest and in transit

For full GDPR compliance, I'd add:

- Cookie consent banner (though we only use HttpOnly auth cookies, not tracking cookies)
 - Automated data deletion pipeline (delete all user data after account deletion + retention period)
 - Data Processing Agreement with OpenAI
 - Record of processing activities (ROPA documentation)
-

Q: What guardrails do you have for a fintech application?

A: Guardrails at every layer:

Data accuracy:

- `Decimal(15,2)` for all monetary amounts — no floating-point errors
- Write-time currency conversion — financial reports are immutable once generated
- Zod schemas reject non-numeric amounts, negative budgets, invalid dates

Operational safety:

- Deletion protection on Aurora (can't accidentally drop the database)
- Soft-delete for users (data isn't permanently lost)
- AuditLog model tracks critical operations

AI guardrails:

- **Schema validation on LLM output** — AI responses must match a Zod schema with required fields (category, suggestion, estimatedSavings). If the LLM returns garbage, it's rejected
- **Deterministic fallback** — rule-engine generates recommendations if the LLM fails. Users always get useful advice
- **No auto-execution** — AI recommendations are INFORMATIONAL ONLY. The system never automatically moves money, deletes transactions, or changes budgets based on AI output

- **Priority-based recommendations** — rule engine assigns priorities (HIGH/MEDIUM/LOW) based on actual spending ratios, not LLM confidence
- **Human review** — recommendations have status (PENDING → ACCEPTED/DISMISSED). User decides what to act on

Error containment:

- One failed CSV row doesn't fail the entire import (partial success model)
- One failed exchange rate provider doesn't block transactions (fallback chain)
- One failed AI call doesn't block the dashboard (dashboard works independently)

8. Financial Data Guardrails

Q: How do you ensure monetary calculations are accurate?

A:

1. **Storage:** `Decimal(15,2)` in PostgreSQL — exact decimal arithmetic
2. **ORM:** Prisma returns Decimal values as strings to avoid JavaScript float precision loss
3. **Conversion:** Currency conversion happens ONCE at write time and the result is stored
4. **Aggregation:** `SUM(baseCurrencyAmount)` in SQL, which operates on Decimal columns
5. **Display:** Amounts formatted with `toFixed(2)` only at the API response boundary

What could go wrong without this:

```
// BAD: floating-point aggregation
[0.1, 0.2, 0.3].reduce((a, b) => a + b) // = 0.6000000000000001

// GOOD: Decimal aggregation in PostgreSQL
SELECT SUM(base_currency_amount) FROM transactions // = 0.60 (exact)
```

Q: What happens if an exchange rate API is down?

A: The `FallbackRateProvider` chains three sources:

1. **API provider** → calls external exchange rate API
2. **Database provider** → queries most recent rate from `ExchangeRate` table
3. **Fixed provider** → hardcoded rates as absolute last resort

Each provider is tried in order. If one fails, the next is tried. The user gets the best available rate without any error message.

Cache layer: The `CurrencyService` also has an **in-memory cache with 1-hour TTL**. Most requests never hit the API at all — they use the cached rate from the last hour.

Q: How do you handle currency conversion edge cases?

A:

- **Same currency** — if transaction currency equals user's base currency, `baseCurrencyAmount = amount`, `exchangeRate = 1.0`. No API call needed
- **Unsupported currency** — Zod schema validates against a whitelist: `["USD", "EUR", "GBP", "INR", "JPY", "CAD", "AUD", "CHF"]`. Unlisted currencies are rejected at validation

- **Rate staleness** — 1-hour cache TTL is acceptable for personal finance. If I needed real-time rates (trading app), I'd use a WebSocket feed
 - **Rate at zero or negative** — would indicate API error. Fixed provider ensures non-zero rates
-

9. AI/ML Integration

Q: How does your AI recommendation engine work?

A: Six-step pipeline:

1. PREPROCESS → Fetch 90 days of transactions for the user
2. AGGREGATE → Group by category, calculate totals and percentages
3. TREND DETECT → Compare current month vs previous months (increasing/decreasing/stable)
4. PROMPT BUILD → Construct structured prompt with spending data + user context
5. LLM CALL → Send to OpenAI GPT-4o-mini, parse JSON response
6. PERSIST → Validate with Zod, save to AIRecommendation table

If step 5 fails (timeout, rate limit, invalid JSON): → Rule engine activates. Same input data, deterministic rules:

- If dining > 30% of income → "Reduce dining out"
 - If subscriptions have recurring charges → "Review subscriptions"
 - If income > expenses by > 20% → "Increase savings allocation"
-

Q: Why GPT-4o-mini over GPT-4 or a self-hosted model?

A:

- **Cost:** \$0.002 per recommendation call (1,750 tokens). GPT-4 would be ~\$0.06 — 30x more expensive
 - **Speed:** GPT-4o-mini responds in ~1-2 seconds. GPT-4 takes 5-10 seconds
 - **Quality:** For structured financial advice generation, GPT-4o-mini is sufficient. I'm not doing complex reasoning — I'm asking it to analyze spending patterns and suggest savings
 - **Self-hosted:** Models like Llama 2 would require a GPU instance (\$0.50+/hour). My entire app costs \$40/month on AWS
-

Q: How do you validate AI output?

A:

```
const schema = z.array(z.object({
  category: z.string(),
  suggestion: z.string(),
  estimatedMonthlySavings: z.number().nonneg(),
  priority: z.enum(["HIGH", "MEDIUM", "LOW"]),
  actionItems: z.array(z.string())
}));

const parsed = schema.safeParse(llmResponse);
if (!parsed.success) {
  // LLM returned invalid structure → fall back to rule engine
```

```
    return generateRuleBasedRecommendations(data);  
}
```

What I validate:

- Response is a JSON array (not a string, not an object)
- Each recommendation has required fields
- `estimatedMonthlySavings` is a non-negative number (LLM can't suggest negative savings)
- `priority` is one of exactly three values
- `actionItems` is a string array

Why this matters: LLMs are non-deterministic. The same prompt can produce different JSON structures. Zod validation ensures only well-formed recommendations reach the user.

Q: How do you handle AI hallucinations in financial recommendations?

A:

1. **Bounded context** — the prompt includes ONLY the user's actual spending data. The LLM can't reference information it doesn't have
 2. **Schema validation** — if the LLM invents a category the user doesn't have, I match against actual category names
 3. **Estimated savings sanity check** — recommendations can't suggest saving more than the user actually spends in that category
 4. **User verification** — all recommendations are PENDING until the user accepts or dismisses them. Nothing is auto-executed
 5. **Deterministic fallback** — worst case, the user gets rule-based recommendations with correct numbers from their actual data
-

10. CSV Import & Data Ingestion

Q: How does CSV import work end-to-end?

A:

1. **Upload:** user sends CSV via `POST /api/transactions/import` with `multipart/form-data`
2. **Multer** receives file in memory (no disk writes)
3. **csv-parser** streams the buffer row-by-row
4. **Each row:**
 - Validated with Zod (amount is number, date is valid, etc.)
 - Category resolved: match by name → use existing ID, or create new category
 - Currency converted using `CurrencyService` (write-time conversion)
 - Transaction created in database
5. **Response:** `{ imported: 45, skipped: 3, errors: [{row: 12, reason: "..."}] }`

Key design choices:

- **Streaming** — `csv-parser` processes rows as they arrive, constant memory usage regardless of file size
- **Partial success** — one bad row doesn't fail the entire import. Good rows are committed, bad rows are reported

- **Category auto-creation** — if the CSV has a category "Coffee Shops" that doesn't exist, it's created automatically
 - **Idempotency** — duplicate detection prevents re-importing the same transactions
-

Q: What if a CSV has 100,000 rows?

A: Currently, it would work but be slow — each row triggers a database query (category lookup + transaction insert). For 100K rows at ~5ms per row, that's ~8 minutes blocking the request.

How I'd fix it:

1. **Batch inserts** — collect 500 rows, use `prisma.transaction.createMany()` for bulk insert
 2. **Queue-based processing** — upload goes to S3, SQS message triggers a Lambda/ECS task. API returns immediately with a job ID. Client polls for completion
 3. **Progress tracking** — store job status in DB: `{ status: "processing", progress: "45000/100000" }`
 4. **Transaction batching** — wrap every 1000 rows in a database transaction for atomicity and performance
-

11. Multi-Currency Handling

Q: How does multi-currency work in your app?

A:

User setup: Each user has a `baseCurrency` (e.g., "INR") set during registration.

Transaction creation:

```
User input: amount=100, currency="EUR"
System:
  1. CurrencyService.convert("EUR", "INR", 100)
  2. Provider chain: cache → API → DB → fixed rates
  3. Result: rate=93.50, baseCurrencyAmount=9350.00
  4. Store: { amount: 100, currency: "EUR", baseCurrencyAmount: 9350.00, exchangeRate: 93.50 }
```

Dashboard/Reports: All queries use `baseCurrencyAmount`. No currency conversion at read time.

Why this design:

- Dashboard `SUM()` queries are simple and fast
 - Reports are deterministic (January total is January total forever)
 - User sees both original and converted amounts on transaction detail
-

Q: What currencies do you support?

A: Eight currencies defined as a TypeScript literal union:

```
type SupportedCurrency = "USD" | "EUR" | "GBP" | "INR" | "JPY" | "CAD" | "AUD" | "CHF";
```

Validated with Zod. Adding a new currency is a one-file change to constants + a Prisma migration to update the enum.

12. Error Handling & Resilience

Q: How do you handle errors across the application?

A: Layered approach:

Layer 1 — Input validation (Zod middleware):

Invalid request → 400 with field-level errors → request never reaches service

Layer 2 — Business logic (typed errors):

```
if (!user) throw new NotFoundError("User", id);           // → 404
if (emailExists) throw new ConflictError("Email taken"); // → 409
if (!bcrypt.compare(...)) throw new AuthError();         // → 401
```

Layer 3 — Global error handler:

AppError subclass? → use its statusCode + message
Unknown error? → 500 + generic message in production, real message in dev

Layer 4 — Process-level:

asyncHandler wraps every route handler — unhandled Promise rejections are caught and forwarded to the error handler instead of crashing the process.

Q: What happens if the database goes down?

A: Currently:

- Prisma's connection pool would exhaust, requests would time out
- Error handler returns 500 to clients
- CloudWatch alarm triggers on ECS error rate, SNS sends email alert

What I'd add for production:

- **Circuit breaker** — after N consecutive failures, stop trying and return a cached response or a circuit-open error (503 with Retry-After)
 - **Health check endpoint** — ALB checks `/health`, marks unhealthy tasks, routes traffic to healthy ones
 - **Read replica** — Aurora supports read replicas. Dashboard queries could use the replica while writes go to primary
-

Q: How do you handle cascading failures?

A: I have isolation between subsystems:

- **AI failure** doesn't block transactions or dashboard. The AI pipeline catches errors and falls back to rule engine
- **Exchange rate failure** doesn't block transactions. FallbackRateProvider ensures a rate is always available

- **Report generation failure** doesn't affect user flows. Reports are generated via cron, failures are logged
- **CSV import failure** is partial — good rows succeed, bad rows are reported

The pattern is: **every external dependency has a fallback**, and **no subsystem failure cascades to the core transaction flow**.

13. Scalability & Performance

Q: Is your application scalable?

A: The architecture is scalable, the current configuration is modest:

Current (small scale):

- 0.25 vCPU, 512 MB per ECS task → ~39 req/sec → ~5-10 concurrent users
- Bottleneck: bcrypt hashing uses 250ms of CPU per login/register

Medium scale (config change):

- Bump to 1 vCPU, 2 GB → 4x throughput → ~50+ concurrent users
- Horizontal scaling: 2-10 ECS tasks behind ALB → ~500 req/sec

Large scale (architecture changes needed):

- Dashboard aggregation: replace on-the-fly queries with materialized views or pre-computed summary tables
- CSV import: move to async queue-based processing
- AI calls: batch and cache recommendations (generate once per day, not on every request)
- Database: read replicas for dashboard, write primary for transactions
- Caching: Redis for session data, exchange rates, and dashboard snapshots

Key principle: The app is **stateless** — no server-side sessions, no in-memory state (except the 1-hour exchange rate cache, which is a performance optimization, not required). This means any number of ECS tasks can handle any request.

Q: What did your stress tests show?

A:

- **Throughput:** 39 requests/sec sustained
- **Latency:** 25ms p95 for API calls, bcrypt-heavy endpoints higher
- **Error rate:** 0% under normal load
- **Breaking point:** 50 concurrent users with login-heavy flows
- **Infrastructure:** 0.25 vCPU, 512 MB, Aurora 0.5 ACU (minimal config)

Honest assessment: The stress test proved the architecture works but the default resources are conservative. Production would need at least 1 vCPU. But there's a clear, tested scaling path with no architectural changes needed up to ~5,000 users.

Q: How does your auto-scaling work?

A: Three dimensions:

ECS (compute):

- Min: 2 tasks, Max: 10 tasks
- Scale triggers:
 - CPU > 70% average
 - Memory > 80% average
 - ALB requests > 1000 per target

Aurora (database):

- Min: 0.5 ACU, Max: 4 ACU
- Scales automatically based on connection count and query load

CloudFront (CDN):

- Infinite scale by design — AWS manages edge capacity
- Static assets served from cache, no origin load

Cooldown: 60 seconds between scale actions to prevent flapping.

Q: How would you handle 1 million users?

A: Phased approach:

Phase 1 (1K-10K users) — optimize current stack:

- Bump ECS to 2 vCPU, 4 GB
- Add Redis for exchange rate cache and dashboard snapshots
- Add database indexes based on slow query logs

Phase 2 (10K-100K users) — add infrastructure:

- Read replicas for dashboard queries
- Queue-based CSV processing (SQS + worker tasks)
- CDN-cached dashboard snapshots (5-minute stale is fine for personal finance)
- Materialized views for monthly aggregations

Phase 3 (100K-1M users) — architectural changes:

- Extract AI service into separate ECS service with its own scaling
 - Database sharding by userId (each shard handles a range of users)
 - Event-driven architecture (EventBridge) for cross-service communication
 - Multi-region deployment for latency
-

14. Frontend Architecture

Q: How is your frontend structured?

A:

```
src/
├─ App.tsx           # Router setup
├─ main.tsx          # React entry + QueryClient provider
├─ pages/            # Route-level components (Dashboard, Transactions, Login)
├─ components/       # Reusable UI (TransactionList, BudgetCard, Chart)
├─ hooks/            # Custom hooks (useTransactions, useDashboard)
```

```
|— lib/           # API client (Axios), utility functions
|— types.ts       # Shared TypeScript types
```

Patterns:

- **Pages** are route-level containers that compose components
- **Components** are presentation-focused, receive data via props or hooks
- **Hooks** encapsulate TanStack Query calls: `useDashboard()` returns `{ data, isLoading, error }`
- **API client** is centralized — Axios instance with base URL, credentials, and interceptors

Q: How do you manage state?

A: Two state categories:

Server state (TanStack Query):

- All API data: transactions, categories, budgets, dashboard, AI recommendations
- Cached with stale-while-revalidate strategy
- Auto-invalidated after mutations
- Background refetch on window focus

Client state (Zustand):

- User preferences, UI state (sidebar, modals, selected filters)
- Minimal — most "state" is actually server state

Why this split? Server state has different concerns: caching, synchronization, staleness, optimistic updates. TanStack Query handles all of this. Zustand handles the small remainder.

Q: How do you handle loading and error states in the UI?

A: TanStack Query provides `isLoading`, `isError`, `error`, `data`, and `isFetching` for every query:

```
const { data, isLoading, error } = useDashboard();

if (isLoading) return <DashboardSkeleton />;
if (error) return <ErrorMessage message={error.message} />;
return <Dashboard data={data} />;
```

Skeleton screens for first load (better UX than spinners). **Background refetch indicator** for subsequent loads. **Error boundaries** for unexpected render errors.

Q: How does the frontend authenticate?

A:

1. Login/register API call → server sets HttpOnly cookies (frontend never touches the token)
2. All subsequent API calls use `axios.defaults.withCredentials = true` — browser automatically includes cookies
3. On page reload, `GET /api/auth/profile` checks if the cookie is still valid
4. If 401, redirect to login page

The frontend **never stores, reads, or manages tokens**. This is the most secure pattern for SPAs.

15. Testing Strategy

Q: How do you test your application?

A:

Unit tests (backend):

- Service layer tests with mocked repositories
- Utility function tests (validation, currency conversion, CSV parsing)
- Error scenario tests (invalid input, missing data)

Integration tests (backend):

- API endpoint tests with real Express app and test database
- Auth flow tests (register → login → protected route)
- CSV import tests with sample files

Stress tests:

- Artillery-based load tests
- Tested throughput, latency percentiles, error rates
- Results documented with specific numbers

What I'd add:

- Frontend component tests with React Testing Library
- E2E tests with Playwright (full user flows)
- Contract tests between frontend and backend API
- Database migration tests (apply → seed → verify → rollback)

Q: How do you test the AI pipeline?

A:

- **Deterministic tests** — rule engine has predictable output given fixed input. I test: "high dining spend → dining reduction recommendation"
- **Schema validation tests** — verify Zod schema catches malformed LLM responses
- **Fallback tests** — mock OpenAI as unreachable, verify rule engine activates
- **Snapshot tests** — save known-good AI recommendations, verify structure doesn't regress

I don't test the LLM output itself (non-deterministic), but I test everything around it: input preparation, output validation, fallback behavior, and persistence.

16. DevOps, CI/CD & Infrastructure

Q: How is your infrastructure managed?

A: Terraform for all AWS resources:

```
infra/terraform/  
├─ vpc.tf           # VPC, subnets, NAT gateway  
├─ security_groups.tf # Network access rules  
└─ rds.tf           # Aurora PostgreSQL Serverless v2
```

```
|— ecs.tf          # ECS cluster, task def, service
|— alb.tf          # Application Load Balancer
|— ecr.tf          # Container registry
|— cloudfront.tf   # CDN + S3 origin
|— autoscaling.tf  # ECS scaling policies
|— monitoring.tf   # CloudWatch alarms + dashboard
|— secrets.tf      # Secrets Manager
|— variables.tf    # Configuration (instance sizes, scaling params)
```

Why Terraform over CloudFormation?

- Multi-cloud portable (not locked to AWS)
- HCL is more readable than JSON/YAML
- Larger community and module ecosystem
- State management lets me see what changed before applying

Why IaC at all?

- Reproducibility — I can destroy and recreate the entire stack identically
- Version control — infrastructure changes are PRs, reviewable
- Documentation — the Terraform files ARE the documentation of what's deployed

Q: Describe your deployment pipeline.

A:

```
Code push → Docker build → Push to ECR → ECS rolling deployment
```

1. **Docker build** — multi-stage: `npm ci` + `npm run build` in builder stage, only production deps + compiled JS in runtime stage
2. **Push to ECR** — tagged with git SHA
3. **ECS deployment** — rolling update (min healthy 100%, max 200%). New tasks start → health check passes → old tasks drain
4. **Circuit breaker** — if the new version fails health checks, ECS automatically reverts to the previous version

Docker optimization:

- Multi-stage build → ~150MB final image (not ~1GB with dev deps)
- `.dockerignore` excludes `node_modules`, tests, local config
- Layer caching — `COPY package*.json` before `COPY . .` so `npm ci` is cached unless deps change

Q: How do you handle environment configuration?

A:

Local development:

- `.env` file with `DATABASE_URL`, `JWT_SECRET`, `OPENAI_API_KEY`, etc.
- `docker-compose.yml` spins up PostgreSQL locally

Production:

- **AWS Secrets Manager** stores all sensitive values

- ECS task definition references Secrets Manager ARNs
- At container startup, ECS injects secrets as environment variables
- No secrets in code, no secrets in Docker images, no secrets in Terraform state

Configuration hierarchy:

```
env.ts validates:
  NODE_ENV (development | production | test)
  PORT
  DATABASE_URL
  JWT_SECRET
  OPENAI_API_KEY
  SMTP credentials
  Exchange rate API key
```

`env.ts` uses Zod to validate all environment variables at startup. If a required variable is missing, the app fails immediately rather than running with undefined values.

17. Monitoring & Observability

Q: How do you monitor your application?

A: Four pillars:

Logs:

- Winston logger with structured JSON output (production)
- Request logger middleware: method, URL, status, duration on every request
- Error logger: full stack traces for 500 errors
- CloudWatch Logs for centralized log aggregation

Metrics:

- ECS Container Insights: CPU, memory, network
- ALB metrics: request count, latency, 5xx rate
- Aurora metrics: connections, CPU, replication lag

Alarms (CloudWatch):

- ECS CPU > 85% for 5 minutes
- ECS Memory > 90% for 5 minutes
- ALB 5xx > 5% for 3 minutes
- ALB latency p99 > 2 seconds
- Aurora CPU > 90% for 10 minutes

Notifications:

- All alarms → SNS topic → email notification

Dashboard:

- 4-panel CloudWatch dashboard: ECS CPU, memory, ALB requests, ALB latency
-

Q: What would you add for production observability?

A:

- **Distributed tracing** — X-Ray or OpenTelemetry. Trace a request from CloudFront → ALB → ECS → Aurora → response
- **APM** — Datadog or New Relic for deeper application performance insights
- **Custom business metrics** — transactions per minute, AI recommendation generation time, CSV import throughput
- **Alerting tiers** — P1 (page immediately: 500 error spike), P2 (Slack alert: high latency), P3 (daily email: slow queries)
- **Log-based metrics** — CloudWatch Metrics from log patterns (e.g., count of "AuthError" per minute)

18. Trade-offs & Honest Limitations

Q: What are the limitations of your current system?

A: (Being honest about limitations shows engineering maturity)

Limitation	Impact	Fix
No rate limiting	Login endpoint vulnerable to brute force	<code>express-rate-limit</code> with Redis store
Single JWT secret	Key leak compromises all tokens	Separate signing keys, rotation policy
No MFA	Single factor only	TOTP integration (speakeasy library)
Synchronous CSV	Large files block the request	Queue-based async processing
On-the-fly aggregation	Dashboard slows with transaction count	Materialized views or summary table
No WebSocket	Dashboard requires manual refresh	WebSocket for real-time updates
No retry logic	External API failures return immediately	Exponential backoff with jitter
No idempotency keys	API is not idempotent for POSTs	Idempotency key header for creates
Single region	US-east-1 only, latency for other regions	Multi-region deployment
No feature flags	Changes require deployment	LaunchDarkly or in-house flag system

Q: If you had more time, what would you build next?

A:

1. **Real-time bank sync** — currently manual. Integrate Plaid for automatic transaction import
2. **Budget alerts** — push notification when spending reaches 80% of budget
3. **Goal tracking** — "Save \$5,000 for vacation by December" with progress tracking
4. **Receipt scanning** — OCR with Tesseract to auto-fill transactions from receipt photos

19. Behavioral & Scenario Questions

Q: A user reports that their dashboard shows the wrong total. How do you debug it?

A:

1. **Check the request** — look at request logs for the user's API call. What parameters? What response?
2. **Query the database** — run `SELECT SUM(baseCurrencyAmount) WHERE userId = ? AND type = 'EXPENSE' AND date BETWEEN ? AND ?`. Does it match?
3. **Check timezone** — user might be in UTC+5:30, dashboard uses UTC. A transaction at 11 PM IST is the next day in UTC
4. **Check currency conversion** — were transactions converted with the correct rate? Look at `exchangeRate` values
5. **Check for deleted/orphaned data** — are there transactions with no category? Were categories reassigned?
6. **Compare DB query with service logic** — run the exact query the `DashboardService` executes

Most likely culprits: timezone mismatch, stale cache, or a recently imported CSV with incorrect `baseCurrencyAmount`.

Q: A user uploaded a 50MB CSV and the app timed out. What do you do?

A:

1. **Immediate fix** — increase request timeout for the import endpoint
 2. **Short-term** — add a file size limit (e.g., 10MB) with clear error message
 3. **Proper solution** — make it asynchronous:
 - Accept the file, store in S3
 - Send message to SQS queue
 - Background worker processes the file
 - API returns `{ jobId: "abc123", status: "processing" }`
 - Client polls `GET /api/import/status/abc123`
 - Worker updates status as it processes: `{ processed: 15000, total: 50000, errors: [...] }`
-

Q: The OpenAI API is returning 429 (rate limited). How do you handle it?

A:

1. **Immediate** — the fallback rule engine activates. Users still get recommendations
 2. **Add retry with exponential backoff** — wait 1s, 2s, 4s, 8s before giving up
 3. **Respect Retry-After header** — OpenAI tells you when to retry
 4. **Cache recommendations** — once generated, save to `AIRecommendation` table. Don't regenerate for the same time period
 5. **Budget management** — track token usage, set per-user daily limits
 6. **Queue** — instead of real-time generation, queue recommendations and process them at a controlled rate
-

Q: How would you migrate from a single-tenant to multi-tenant architecture?

A: FinPilot is already effectively multi-tenant at the application level:

- Every table has `userId` as a foreign key
- Every query is scoped by `userId`
- The repository pattern enforces this consistently

For true multi-tenancy (e.g., companies as tenants):

1. Add a `Tenant` model with `tenantId` on every table
 2. Middleware extracts `tenantId` from subdomain or JWT
 3. All queries add `WHERE tenantId = ?`
 4. Consider database-per-tenant for strict isolation (compliance requirement for some fintech clients)
-

Q: Tell me about a time you made a design decision you later regretted.

A: "Initially I stored all amounts only in the user's base currency, discarding the original amount and currency. This made queries simple but I lost information — users couldn't see their original transaction amount in the currency they paid in. I had to add `amount`, `currency`, and `exchangeRate` columns and migrate existing data. The lesson: **store the raw data and the derived data**, not just the derived data."

Q: How would you approach on-call for this system?

A:

- **Runbook** for common issues (high CPU → scale ECS, DB connections exhausted → check connection pool, AI API down → verify fallback is active)
 - **Tiered alerts** — P1: service down or data corruption. P2: degraded performance. P3: non-critical warnings
 - **Dashboard** — CloudWatch dashboard with all key metrics visible at a glance
 - **Rollback plan** — ECS deployment has circuit breaker. If new version fails, it auto-reverts
 - **Post-mortem** — after every incident, write what happened, why, timeline, fix, and prevention measures
-

20. Intuit-Specific Questions

Q: How does your experience relate to what Intuit does?

A: Intuit builds TurboTax, QuickBooks, Mint, and Credit Karma — all financial data platforms. FinPilot touches:

- **Financial data accuracy** — same concern as TurboTax (amounts must be exact)
- **Multi-currency** — QuickBooks handles international transactions
- **AI-powered insights** — Mint's spending insights, Credit Karma's recommendations
- **Data security** — all Intuit products handle sensitive financial data
- **CSV/data import** — QuickBooks imports bank statements
- **Report generation** — TurboTax generates tax documents

I've dealt with the same fundamental challenges at a smaller scale: exact decimal arithmetic, secure data handling, AI integration with fallbacks, and user-facing financial reports.

Q: Intuit handles millions of transactions during tax season. How would you scale your system for that?

A:

- **Read/write separation** — split to primary (writes) and read replicas
- **Pre-computation** — batch-compute summaries nightly instead of real-time aggregation
- **Caching layers** — Redis for hot data, CDN for static reports
- **Queue everything** — decouple ingestion from processing. Accept transactions into SQS, process async
- **Horizontal partitioning** — shard database by user ID range
- **Feature flags** — gradually roll out to users, control blast radius
- **Auto-scaling with warm pool** — pre-warm ECS tasks before known peak periods

The key insight: tax season is **predictable**. You know when the load comes (January-April), so you can pre-scale rather than react.

Q: How would you handle regulatory compliance at scale?

A:

- **Audit trail** — every financial operation logged with timestamp, user, action, before/after values (I have the AuditLog model for this)
 - **Data residency** — deploy in the user's region (US data in US, EU data in EU)
 - **Retention policies** — automated data lifecycle (keep transactions for 7 years per IRS requirements, anonymize after)
 - **Access logging** — who accessed what financial data, when
 - **Encryption** — at rest (KMS), in transit (TLS), and potentially field-level encryption for SSN/sensitive fields
 - **SOC 2 compliance** — access controls, monitoring, incident response procedures
-

Q: How do you ensure data consistency in a financial application?

A:

1. **Database constraints** — NOT NULL, foreign keys, unique constraints prevent invalid data
 2. **Application validation** — Zod schemas reject bad data before it reaches the database
 3. **Atomic transactions** — related operations (create transaction + update audit log) happen in a single Prisma transaction
 4. **Write-time conversion** — base currency amount is immutable after creation
 5. **Idempotent operations** — CSV import deduplicates by description + date + amount
 6. **Type safety** — TypeScript prevents passing a string where a number is expected
 7. **Decimal arithmetic** — PostgreSQL `Decimal(15,2)` prevents floating-point drift
-

Q: What's your approach to code quality and maintainability?

A:

- **TypeScript strict mode** — no implicit any, strict null checks
- **Layered architecture** — clear separation of concerns means you can modify one layer without affecting others
- **Typed errors** — no generic `throw new Error("something")`. Each error type maps to an HTTP status

- **Validation at boundaries** — input validated once at entry, trusted internally
- **Consistent patterns** — every resource follows the same route → service → repository flow
- **Repository pattern** — if I need to optimize a query, I change one method in one file

Q: Any questions for me? (Flip the script — questions you should ask)

Suggested questions to ask the interviewer:

1. "What does the team's development workflow look like — trunk-based or feature branches?"
 2. "What's the deployment frequency — daily, weekly, continuous?"
 3. "What's the biggest technical challenge the team is currently facing?"
 4. "How does the team handle technical debt — dedicated sprints, boy scout rule?"
 5. "What does the on-call rotation look like for a junior engineer?"
 6. "What's the tech stack? Are there opportunities to contribute to architectural decisions?"
-

Quick Reference — Key Numbers

Metric	Value
Total API endpoints	39
Database models	9
Supported currencies	8
bcrypt salt rounds	12
Access token TTL	15 minutes
Refresh token TTL	7 days
Exchange rate cache TTL	1 hour
AI cost per call	~\$0.002
Stress test throughput	39 req/sec
ECS tasks (min/max)	2/10
Aurora ACU range	0.5-4
Default categories seeded	12
CSV import: partial success	Yes
AI fallback: rule engine	Yes
Infrastructure as Code	Terraform
Docker image size (production)	~150 MB
AWS monthly cost (minimal)	~\$40
